BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an* `__init__` *method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent would need a table containing the best action for every possible situation the agent might face.
Problem: Chess has approximately 10^{47} possible board states - this is more than the number of atoms in the universe
As the agent plays longer: The number of possible situations grows exponentially , The

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules"

discussed in the lecture.

```
# CONDITION-ACTION RULE 1
if food_here:              # CONDITION: Agent senses food
    return 'Stay'          # ACTION: Collect the food

# CONDITION-ACTION RULE 2
if wall_ahead:             # CONDITION: Agent senses wall
    return 'Left'          # ACTION: Turn left

# CONDITION-ACTION RULE 3
if not wall_ahead and not food_here:  # CONDITION: Clear path
    return 'Up'                       # ACTION: Move forward
```

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

2 reasons:
1. Partial Observability:
The agent can only see if there's a wall ahead or food here
It cannot see its exact position in the grid
Different locations look the same to the agent

2. No Memory (No Percept History):
The agent has no memory of past actions
Each decision is made as if it's the first time
It cannot remember: "I already tried turning left"

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

1. Transition Model (Tracking where you've been):
```
self.visited_cells = set()          # Stores visited positions
self.visited_cells.add(current_pos) # Update memory
```

2. Sensor Model (Tracking what you've tried):
```
self.action_history = []           # Stores actions taken
self.action_history.append(action)  # Record what was tried
```

3. Memory-Aware Rules:
```
# IF wall_ahead AND left_is_visited THEN turn_right
if wall_ahead and left_visited:
    action = 'Right'  # Try different action
```

How Model-Based Agent Escapes Loops

The Model-Based Agent uses memory to escape loops. When stuck, it checks if it has been here before. If yes, it remembers what actions it already tried (like turning left) and chooses a different action (like turning right) to break out of the cycle.

Finalize and Lock Lab 02 Documentation

FACULTY OF COMPUTING